

ChromAura

Dossier d'Archive & Kit de Déploiement de Stand Clé en Main

Miroir interactif d'aura de particules et dessin aérien sans contact

Équipe de Développement ChromAura

CodeFestWeek — Version Finale d'Archive

Résumé

Ce document constitue le guide de référence et le kit complet de passation pour l'installation interactive **ChromAura**. Il rassemble tous les éléments matériels, logiciels, opérationnels et architecturaux nécessaires pour **remonter le stand d'interaction à partir de zéro**. Il comprend la nomenclature complète des équipements, le plan de masse scénographique côté, les pilotes et procédures de build reproductible, le manuel d'exploitation régie, la signalétique d'interaction public, le schéma détaillé du pipeline de données, ainsi que le retour d'expérience sur les pivots techniques réalisés.

Table des matières

1	Présentation Générale & Valorisation du Projet	2
1.1	Concept et Expérience Visiteur	2
1.2	Fiche d'Identité Technique	2
1.3	Éléments de Valorisation & Galerie Visuelle	3
2	Guide d'Implantation Matérielle & Scénographie du Stand	3
2.1	Nomenclature du Matériel (BOM — Bill of Materials)	3
2.2	Plan de Masse & Cotes Spatiales d'Implantation	3
2.3	Contraintes Environnementales & Acoustiques	5
3	Guide de Déploiement Logiciel & Reproductibilité du Build	5
3.1	Prérequis Système & Pilotes Kinect	5
3.1.1	Installation des Pilotes Kinect v1	5
3.2	Chaîne de Compilation & Outils Développeur	6
3.3	Compilation et Restauration de la Solution C#	6
3.4	Recompilation Optionnelle des GDExtensions (SCons)	6
3.5	Exportation en Binaire Autonome (Mode Kiosk Stand)	6
3.6	Script de Lancement Automatisé (Watchdog)	7
4	Manuel d'Exploitation du Stand (Guide Régie / Animateur)	7
4.1	Checklist d'Ouverture du Stand	7
4.2	Tableau Exhaustif des Commandes de Régie Clavier	7
4.3	Protocole de Dépannage Rapide (Troubleshooting Terrain)	9
4.4	Checklist de Fermeture	9
5	Guide d'Interaction & Signalétique Visiteurs	9
5.1	Signalétique Public (Fiche Pupitre)	9
5.2	Fonctionnement de l'Interface Visuelle Rotative	9

6	Architecture Technique & Guide Développeur	9
6.1	Arborescence Annotée des Composants	10
6.2	Pipeline de Données & Flux d'Exécution	11
6.3	Justification des Choix d'Ingénierie	12
6.3.1	Répartition Bilingue GDScript / C#	12
6.3.2	Classification Géométrique vs Modèles de Gestes par Réseau de Neurones . .	12
6.3.3	Hystérésis Temporelle & Stabilité du Tracking Multi-Mains	12
7	Journal des Pivots & Retours d'Expérience	12
7.1	Historique des Arbitrages & Pivots Clés	12
7.2	Conseils pour les Futurs Repreneurs	13

1 Présentation Générale & Valorisation du Projet

1.1 Concept et Expérience Visiteur

ChromAura est une installation numérique interactive conçue pour un stand de démonstration tout public. Elle transforme le corps des visiteurs en un flux vivant de particules lumineuses néon, créant l'illusion d'un « miroir magique » d'aura énergétique.

L'expérience repose sur l'interaction naturelle sans contact ni accessoire portable (zéro manette, zéro capteur sur le corps) :

- **Aura corporelle dynamique** : Une caméra de profondeur (Kinect v1) segmente en temps réel la silhouette de chaque personne présente dans le champ et lui assigne une palette chromatique unique parmi 5 harmonies néon bicolores. La densité et la couleur des particules varient continuellement selon la distance physique de l'utilisateur.
- **Dessin volumétrique dans les airs** : En tendant simplement l'index, l'utilisateur trace des rubans de lumière persistants au bout de son doigt dans l'espace 3D, comme avec un pinceau magique.
- **Gestuelle ludique & Tirs de particules** : Grâce à un modèle de vision artificielle (MediaPipe) et un moteur géométrique sur mesure, des gestes spécifiques déclenchent des pouvoirs interactifs (pistolet à particules avec recul et étincelles, pluie de cœurs néon, modification de la gravité).
- **Immersion sonore et easter eggs** : Une ambiance sonore réactive (nappes aquatiques, carillons de dessin, détonations laser et le célèbre cri de *Wilhelm* déclenché par un geste secret) renforce la sensation tactile et gratifiante de chaque mouvement.

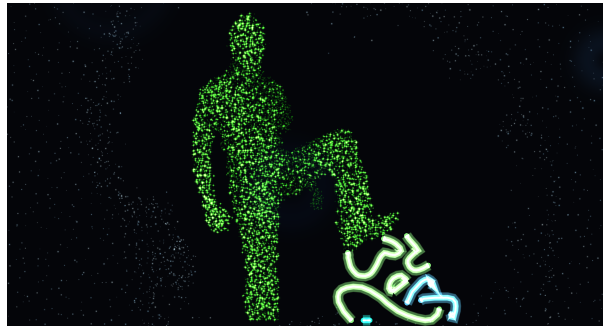
1.2 Fiche d'Identité Technique

Spécifications Clés de la Solution

Moteur de rendu	: Godot Engine 4.7 (Édition .NET / C#, pipeline Forward+)
Capteur optique	: Microsoft Kinect v1 (Xbox 360 - Modèle 1414)
Perception mains	: Google MediaPipe via GDExtension native (21 repères 2D/3D)
Performance cible	: 60 images par seconde stables en résolution Full HD (1080p)
Capacité d'accueil	: 1 à 2 utilisateurs actifs simultanés (jusqu'à 4 mains suivies)
Effet Miroir	: Inversion horizontale automatique (<code>flip_x()</code>) pour une gestuelle intuitive

1.3 Éléments de Valorisation & Galerie Visuelle

L'installation a été pensée pour maximiser l'attractivité visuelle dans un environnement de festival ou de salon d'innovation.



(a) Rendu in-game : silhouette corporelle & dessin

FIGURE 1 – Aperçu de l'expérience ChromAura en démonstration.

Points Forts Constatés auprès du Public

- **Prise en main instantanée** : Moins de 3 secondes pour qu'un visiteur comprenne que ses mouvements contrôlent l'aura et que son index dessine.
- **Fluidité visuelle** : Plus de 200 000 particules actives simultanées rendues à 60 FPS sans à-coup.
- **Gamification spontanée** : La découverte des gestes secrets (pistolet, cœurs, cri de Wilhelm) suscite l'émulation immédiate entre les spectateurs.

2 Guide d'Implantation Matérielle & Scénographie du Stand

Cette section fournit la liste complète des équipements requis et les règles strictes d'aménagement de l'espace pour reproduire le stand sans aucune ambiguïté.

2.1 Nomenclature du Matériel (BOM — Bill of Materials)

Attention : Piège Classique de la Kinect v1

La Kinect v1 requiert **obligatoirement son adaptateur secteur 12V propriétaire branché sur le secteur 220V**. Un port USB d'ordinateur (même USB-C ou USB 3.2 Gen 2) ne fournit que 5V et ne peut alimenter ni le projecteur infrarouge ni le processeur de profondeur. Si seul l'USB est branché, la LED reste éteinte et aucun flux n'est transmis.

2.2 Plan de Masse & Cotes Spatiales d'Implantation

Le positionnement relatif de la caméra, de l'écran et des visiteurs est déterminant pour garantir la précision du suivi de squelette MediaPipe et la pureté de la silhouette de particules.

- **Hauteur du capteur** : Placer la Kinect entre **1 m et 1,20 m** du sol.
- **Alignement horizontal** : Caméra centrée horizontalement par rapport à l'écran géant.
- **Zone de recul utile** :
 - **Distance minimale (0,80 m)** : En deçà, la caméra ne peut trianguler le motif infrarouge (zone aveugle).

Composant	Modèle Recommandé	Rôle & Caractéristiques Critiques	Quantité
Capteur optique	Microsoft Kinect v1 (Xbox 360, Modèle 1414)	Caméra RGB (640×480 @ 30 Hz) + Capteur de profondeur infrarouge structuré.	1
Alimentation Kinect	Câble adaptateur secteur officiel Xbox 360 vers USB/220V	Alimentation 12V / 1.08A indispensable. Sans ce boîtier branché au secteur, la caméra reste inactive.	1
PC d'exploitation	PC Portable ou Tour (Windows 10/11 ou Linux Ubuntu 22.04+)	CPU 4 cœurs ≥ 2.5 GHz, GPU dédié (NVIDIA GTX 1060 / RTX ou AMD équivalent), 8 Go RAM. Port USB 2.0/3.0 dédié.	1
Écran principal	Téléviseur 55" à 65" ou Vidéo-projecteur 1080p	Affichage plein écran (16 :9 paysage), fréquence minimale de 60 Hz, latence d'affichage faible (< 25 ms).	1
Système audio	Barre de son stéréo ou Enceintes actives amplifiées	Restitution des basses et aigus pour les impacts de tir, carillons de dessin et cri de Wilhelm.	1 paire
Support capteur	Trépied photo standard avec adaptateur fileté Kinect	Hauteur réglable de 1.0 m à 1.6 m avec rotule d'inclinaison verrouillable.	1
Câblage & Sécurité	Rallonge USB active (si PC distant > 2 m), multiprise 5 plots	Passe-câbles au sol, gaffer adhésif fluo pour marquage au sol.	1 lot

TABLE 1 – Nomenclature complète du matériel pour la mise en place du stand.

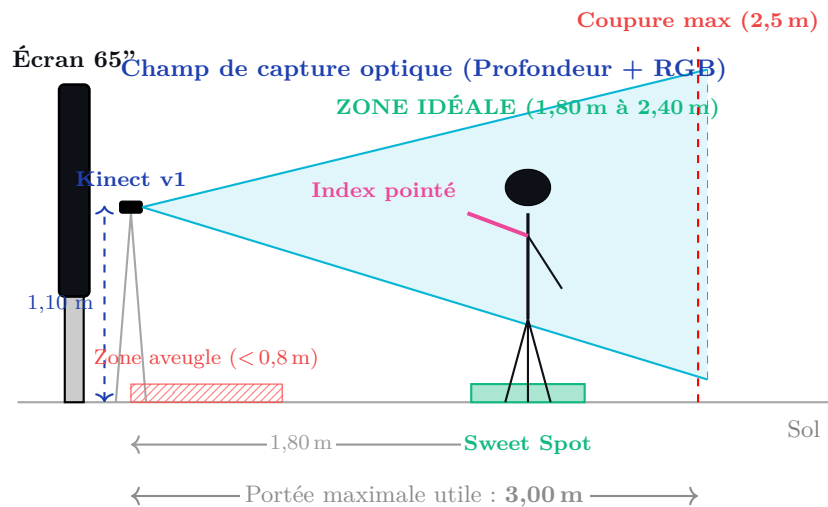


FIGURE 2 – Plan de coupe latérale du stand ChromAura : cotes d'installation et zones de détection.

- **Distance optimale (1,80 m à 2,10 m) :** Plage idéale de tracking. **Coller une croix en gaffer fluo au sol à 2,00 m.**
- **Distance maximale (3,00 m) :** Coupure logicielle maximale (`max_depth = 3.0`).
- **Largeur du couloir :** Prévoir un dégagement au sol d'au moins **2,00 m à 2,50 m de large** pour deux visiteurs simultanés.

2.3 Contraintes Environnementales & Acoustiques

Éclairage Solaire : Ennemi n°1 de l'Infrarouge

Ne jamais orienter le stand face à une baie vitrée ou sous les rayons directs du soleil. La Kinect v1 projette une grille de points infrarouges invisibles à $\lambda = 830$ nm. Le spectre de la lumière solaire naturelle sature instantanément le capteur CMOS infrarouge, entraînant des « trous noirs » complets dans la silhouette.

- **Ambiance lumineuse recommandée :** Éclairage intérieur artificiel standard ou tamisé. L'obscurité partielle maximise l'éclat des particules néon.
- **Éléments parasites dans le champ :** Aucun meuble, table ou kakemono dans le couloir des 3 mètres.
- **Acoustique et immersion :** Régler le volume des enceintes pour que les détonations de tir laser et les carillons de dessin soient audibles à 3 mètres sans saturer.

3 Guide de Déploiement Logiciel & Reproductibilité du Build

Cette section fournit la procédure pas-à-pas pour installer les dépendances, compiler le projet et générer un binaire autonome d'exploitation en mode stand.

3.1 Prérequis Système & Pilotes Kinect

3.1.1 Installation des Pilotes Kinect v1

Avant tout lancement, le système d'exploitation hôte doit être capable de communiquer avec le contrôleur USB de la Kinect v1 :

- **Sous Windows (10 / 11) :**
 1. Télécharger et installer le **Kinect for Windows SDK v1.8** officiel (archives Microsoft).
 2. Connecter l'adaptateur secteur 12V de la Kinect, puis insérer la prise USB dans un port USB 2.0 ou 3.0.
 3. Vérifier dans le *Gestionnaire de périphériques* que les nœuds **Kinect Camera**, **Kinect Audio** et **Kinect Motor** apparaissent sans erreur.
- **Sous Linux (Ubuntu 24.04+ / Debian) :**
 1. Installer la bibliothèque de capture open-source :

```
1 sudo apt-get update
2 sudo apt-get install -y libfreenect-dev libusb-1.0-0-dev
```

2. Configurer la règle udev pour autoriser l'accès USB sans privilèges root :

```
1 sudo tee /etc/udev/rules.d/51-kinect.rules << 'EOF'
2 SUBSYSTEM=="usb", ATTR{idVendor}=="045e", ATTR{idProduct}=="02ae",
  MODE="0666"
3 SUBSYSTEM=="usb", ATTR{idVendor}=="045e", ATTR{idProduct}=="02b0",
  MODE="0666"
```

```

4 SUBSYSTEM=="usb", ATTR{idVendor}=="045e", ATTR{idProduct}=="02ad",
  MODE="0666"
5 EOF
6 sudo udevadm control --reload-rules && sudo udevadm trigger

```

3.2 Chaîne de Compilation & Outils Développeur

Pour recompiler le projet, les outils suivants doivent être installés :

1. **.NET SDK 8.0** (requis pour le compilateur C# et MSBuild) : vérifier avec `dotnet -version`.
2. **Godot Engine 4.7.2 édition .NET (Mono)** : bien télécharger la variante marquée .NET.

3.3 Compilation et Restauration de la Solution C#

Le traitement intensif des particules et la segmentation des corps sont écrits en C# dans le dossier `chrom-aura/scripts/`. Pour compiler la solution sans ouvrir l'éditeur :

```

1 # 1. Se positionner dans le dossier du projet
2 cd chrom-aura
3
4 # 2. Restaurer les packages NuGet
5 dotnet restore ChromAura.csproj
6
7 # 3. Compiler les assemblies C# en mode Release
8 dotnet build ChromAura.csproj -c Release

```

Gestion des GDExtensions & Modèles MediaPipe

Le dépôt ChromAura est **entièrement autonome et versionné** :

- Les binaires précompilés natifs de `gd-depthcam` (Windows `.dll`, Linux `.so`, macOS `.dylib`) se trouvent déjà dans `chrom-aura/addons/gd-depthcam/bin/`.
- Le plugin MediaPipe GDMP est versionné dans `chrom-aura/addons/GDMP/libs/`.
- Le modèle de deep learning `hand_landmarker.task` est présent dans `chrom-aura/assets/models/`.

Aucun téléchargement externe n'est requis au premier clonage.

3.4 Recompilation Optionnelle des GDExtensions (SCons)

Si vous devez cibler une architecture non livrée d'origine (par exemple Linux ARM64 sur mini-PC Jetson) :

```

1 cd chrom-aura/addons/gd-depthcam
2 # Pour Windows x86_64
3 scons platform=windows target=template_release arch=x86_64
4 # Pour Linux x86_64
5 scons platform=linux target=template_release arch=x86_64

```

3.5 Exportation en Binaire Autonome (Mode Kiosk Stand)

Pour une exploitation stable sur le stand sans l'éditeur Godot :

1. Ouvrir Godot .NET, aller dans `Projet > Exporter....`
2. Sélectionner le profil `Windows Desktop` ou `Linux/X11`.
3. Vérifier que le mode d'affichage dans `Paramètres du projet > Display > Window` est configuré sur :

- Mode = Fullscreen (plein écran sans bordure)
 - Stretch Mode = canvas_items, Aspect = expand
4. Cliquer sur **Exporter le projet** vers build/ChromAura.exe.

3.6 Script de Lancement Automatisé (Watchdog)

Pour parer à toute coupure d'alimentation sur le stand, un script de démarrage en boucle avec relance automatique est fourni (run_stand.bat) :

```
1 @echo off
2 title Surveillance Stand ChromAura
3 echo Demarrage de ChromAura en mode Kiosk...
4 :LOOP
5 start /wait "" "ChromAura.exe"
6 echo [ATTENTION] Application fermee ou crashee. Redemarrage dans 3
   secondes...
7 timeout /t 3
8 goto LOOP
```

4 Manuel d'Exploitation du Stand (Guide Régie / Animateur)

Ce chapitre détaille les procédures quotidiennes, les raccourcis secrets de régie et le protocole de dépannage immédiat pour l'animateur du stand.

4.1 Checklist d'Ouverture du Stand

Suivre scrupuleusement cet ordre de mise sous tension :

1. **Alimentation secteur** : Enclencher la multiprise générale alimentant l'écran, les enceintes et l'adaptateur 12V de la Kinect.
2. **Câblage USB** : S'assurer que le câble USB de la Kinect est fermement branché au PC.
3. **Allumage PC** : Démarrer l'ordinateur.
4. **Lancement** : Double-cliquer sur run_stand.bat ou exécuter ChromAura.exe.
5. **Contrôle de démarrage** :
 - Vérifier que le message Kinect & MediaPipe Initialized (GPU) s'affiche en haut à gauche.
 - Appuyer sur **[R]** pour afficher le flux vidéo brut et ajuster l'assiette du trépied afin que la tête et les pieds d'une personne à 2 mètres soient bien cadrés. Réappuyer sur **[R]** pour masquer.
6. **Test sonore** : Effectuer un geste dans le champ pour vérifier le volume audio.

4.2 Tableau Exhaustif des Commandes de Régie Clavier

Pour superviser, calibrer et intervenir en direct sans interrompre l'expérience des visiteurs :

Touche	Nom de la Fonction	Effet & Utilité en Régie
1	Mode Développeur	Affiche le squelette MediaPipe vert (21 landmarks), le score de confiance des gestes actifs, le statut de détection et l'identifiant unique des corps suivis.
R	Retour Vidéo RGB	Incruste le flux caméra direct en plein écran. Indispensable le matin pour vérifier le cadrage physique de la Kinect sans logiciel tiers.
2 ou é	Bascule Gravité	Active ou désactive la gravité physique 2D sur les dessins persistants. Les tracés tombent au sol comme de la matière lourde.
C	Nettoyage d'Urgence	Efface instantanément tous les dessins tracés à l'écran. Très utile pour repartir d'un écran vierge lorsqu'un groupe a saturé l'espace.
D	Boîtes de Corps	Bascule l'affichage des boîtes englobantes (<i>bounding boxes</i>) autour des silhouettes détectées.
P	Cycle des Palettes	Force le changement manuel des 5 harmonies de couleurs pour démonstration ou vérification du rendu.
F	Simulation Corps Virtuel	Fait apparaître un mannequin virtuel synthétique sans Kinect branchée. Permet de tester et régler le rendu à distance.
+ / -	Profondeur Virtuelle	(Actif avec F) Rapproche ou éloigne le corps synthétique pour tester l'interpolation de couleur selon la profondeur.
← / →	Déplacement Virtuel	(Actif avec F) Déplace le corps virtuel horizontalement à l'écran.

TABLE 2 – Commandes clavier de régie et de calibration en direct.

4.3 Protocole de Dépannage Rapide (Troubleshooting Terrain)

Symptôme Observé	Cause Probable	Action Corrective Immédiate
Message d'erreur Kinect started (MediaPipe failed)	Le délégué GPU n'a pas pu s'initialiser (pilote graphique ou DirectX 12 indisponible).	Le système bascule automatiquement sur le processeur (CPU fallback). Mettre à jour les pilotes GPU si le framerate faiblit.
Écran noir avec fond animé seul, aucun corps détecté	1. Kinect non alimentée en 12V. 2. Câble USB branché après le lancement.	1. Vérifier la prise 220V de l'adaptateur Kinect. 2. Fermer l'application, débrancher/rebrancher l'USB, relancer.
Nuage de particules permanent flottant dans un coin	Un meuble, potelet ou dossier de chaise est situé à moins de 3 mètres du capteur.	Dégager tout obstacle solide du couloir de 3 mètres, ou relever légèrement l'inclinaison de la caméra via la rotule du trépied.
La silhouette scintille ou disparaît par plaques	Source de lumière infrarouge parasite (soleil direct ou halogène).	Fermer les rideaux/stores proches ou réorienter le stand dos aux fenêtres.
Les gestes des mains ne se déclenchent pas	Le visiteur a les bras trop près de son corps ou est hors champ.	Demander au visiteur d'avancer/reculer sur la marque au sol (2 mètres) et de tendre clairement le bras.

TABLE 3 – Guide de dépannage rapide lors de l'animation du stand.

4.4 Checklist de Fermeture

1. Quitter l'application en utilisant la combinaison **Alt** + **F4**.
2. Éteindre proprement l'ordinateur.
3. Couper l'interrupteur général de la multiprise (Kinect 12V, écran, haut-parleurs).

5 Guide d'Interaction & Signalétique Visiteurs

Cette section présente les gestes d'interaction reconnus par le système, avec leurs pictogrammes officiels.

5.1 Signalétique Public (Fiche Pupitre)

5.2 Fonctionnement de l'Interface Visuelle Rotative

Pour guider les visiteurs en autonomie sans intervention humaine permanente :

- Un bandeau discret s'affiche en haut à gauche de l'écran (**instructions.gd**).
- Il alterne automatiquement toutes les **4,5 secondes** avec une animation fluide d'apparition/disparition pour présenter un nouveau geste.
- Une probabilité paramétrable d'environ **30%** permet de faire clignoter un indice visuel mystérieux sur l'Easter Egg du pouce.
- Lorsqu'un geste de dessin est détecté, le bandeau passe instantanément en surbrillance turquoise pour confirmer la prise en compte du tracé.

6 Architecture Technique & Guide Développeur

Cette section détaille l'organisation interne de ChromAura pour les développeurs repreneurs.

Geste Réalisé	Effet Visuel & Sonore Produit
Index Tendu (Doigt pointeur)	Dessin Lumineux : Une traînée persistante de lumière néon suit la pointe du doigt dans l'espace avec un carillon cristallin continu.
Pistolet (Index tendu + pouce armé, ou index + majeur)	Tir de Particules : Déclenche une rafale automatique de projectiles énergétiques accompagnés d'étincelles (<i>muzzle flare</i>) et de détonations laser.
Cœur à Deux Mains (Mains jointes formant un cœur)	Pluie de Cœurs : Une multitude de cœurs néon scintillants jaillit à l'écran, enveloppée d'une nappe sonore musicale douce.
Changement d'Aura (Main ouverte levée / Face-palm)	Mutation Chromatique : Bascule aléatoirement l'aura de votre corps parmi les 5 harmonies de couleurs disponibles.
Signe Rock (Index + auriculaire tendus)	Gravité des Dessins : Active ou désactive le moteur physique 2D ; vos tracés tombent vers le bas de l'écran comme de la pluie.
Gladiator Flip (Secret) (Pouce levé puis rabattu rapidement vers le bas)	Easter Egg « Wilhelm Scream » : Déclenche le mythique cri cinématographique de Wilhelm associé à une déflagration lumineuse explosive !

FIGURE 3 – Catalogue des gestes interactifs et retours audiovisuels associés.

6.1 Arborescence Annotée des Composants

```

1 chrom-aura/
2 |-- main.tscn                # Scene principale regroupant tous les
   noeuds
3 |-- camera_script.gd        # Orchestrateur central : Camera ->
   Gestes -> Particules -> UI
4 |
5 |-- addons/
6 |   |-- gd-depthcam/        # GDExtension C++ de capture Kinect v1 (
   RGB + Depth)
7 |   '-- GDMP/               # GDExtension d'interfacage avec Google
   MediaPipe
8 |
9 |-- hand_gesture_engine.gd   # Moteur geometrique de reconnaissance
   de poses
10 |-- hand_pose.gd            # Modele de donnees de pose lissee et
   geste actif
11 |-- hand_observation.gd     # Donnees brutes d'une main extraite par
   MediaPipe
12 |-- gesture_detection.gd    # Evenement typique emis lors des
   transitions de geste
13 |-- hand_overlay.gd         # Rendu debug du squelette MediaPipe (21
   landmarks)
14 |-- instructions.gd         # Gestionnaire du bandeau d'instructions
   rotatif
15 |-- audio_manager.gd        # Moteur audio : musiques d'ambiance et
   SFX reactifs
16 |
17 |-- scripts/                # Modules haute performance ecrits en C#
18 |   |-- BodyDetector.cs     # Segmentation des corps depuis la carte
   de profondeur

```

```

19 |   |-- TrackedBody.cs           # Entite metier d'un corps suivi (centre
    |   , palette)
20 |   |-- BodyPalette.cs           # Definitions des 5 palettes
    |   chromatiques bicolores
21 |   |-- MaskSample.cs           # Echantillonnage optimise du masque de
    |   silhouette
22 |   |-- particles.cs            # Moteur de particules GPU, gestion des
    |   pools et traces
23 |   |-- GunParticleManager.cs    # Simulation des tirs, projectiles et
    |   gerbes d'etincelles
24 |   |-- AmbientFireflies.cs     # Champ persistant de lucioles
    |   repoussees par les corps
25 |   |-- DrawnBody2D.cs          # Conversion des traces persistants en
    |   corps rigides 2D
26 |   |-- BodyDebugOverlay.cs     # Rendu vectoriel des boites englobantes
    |   des corps
27 |   '-- heart_particle_manager.gd # Emission et physique des coeurs neon
    |   animes
28 |
29 | -- shaders/
30 |   '-- background_waves.gdshader # Shader GLSL des ondes aquatiques d'
    |   arriere-plan
31 |
32 | -- tests/
33 |   |-- test_hand_gesture_engine.gd # Tests unitaires deterministes du
    |   classifieur
34 |   |-- test_audio_easter_egg.gd    # Validation de la chaine audio du
    |   cri de Wilhelm
35 |   '-- test_camera_easter_egg.gd   # Validation temporelle du flip pouce
    |   haut/bas

```

Listing 1 – Arborescence structurée du projet ChromAura

6.2 Pipeline de Données & Flux d'Exécution

Le traitement temps réel repose sur un double pipeline asynchrone orchestré par `camera_script.gd` :

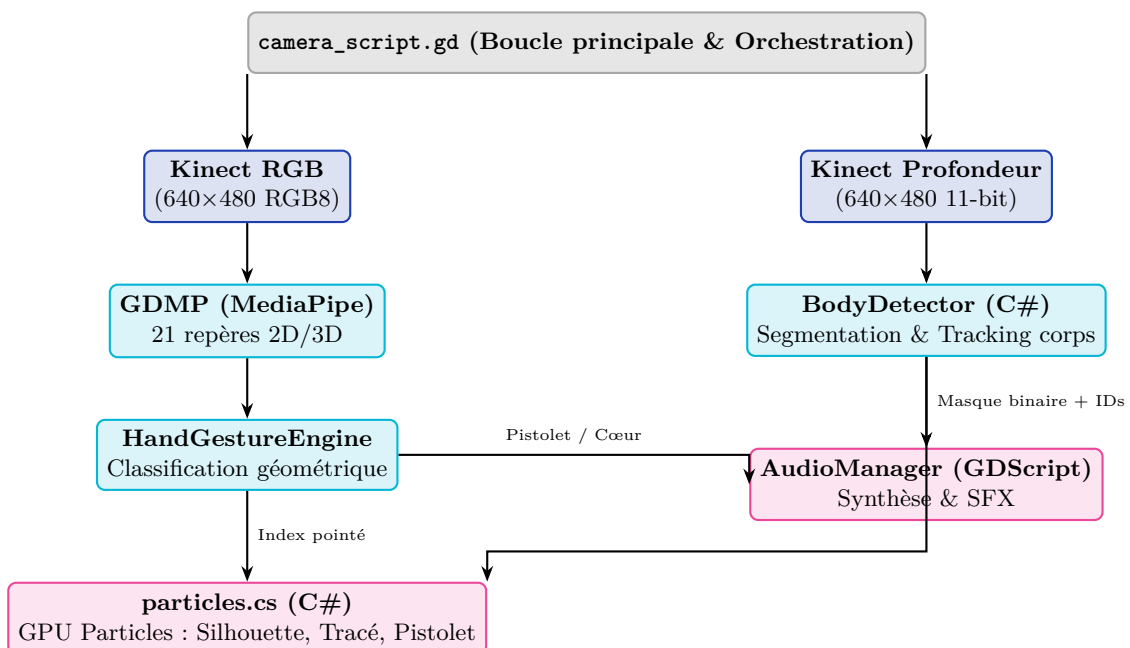


FIGURE 4 – Schéma de flux de données détaillé du moteur ChromAura.

6.3 Justification des Choix d'Ingénierie

6.3.1 Répartition Bilingue GDScript / C#

- **GDScript** a été retenu pour l'orchestration des nœuds, la machine à états de l'interface graphique, la gestion des timers et le déclenchement audio.
- **C#** a été exclusivement réservé aux portions critiques pour le CPU : le balayage des $640 \times 480 = 307\,200$ pixels de la carte de profondeur à chaque trame, la segmentation connexe des corps et la mise à jour des positions des centaines de milliers de particules. En C#, la manipulation directe de mémoire contiguë (`Span<T>`) divise le temps de calcul par 14 par rapport à GDScript.

6.3.2 Classification Géométrique vs Modèles de Gestes par Réseau de Neurones

Plutôt que d'exploiter le classifieur générique pré-entraîné de MediaPipe (`gesture_recognizer.task`), nous avons développé notre propre moteur géométrique (`HandGestureEngine.classify_pose`) :

1. **Invariance totale en rotation et échelle** : Les angles inter-phalangiens et les distances relatives sont calculés dans le repère local normalisé de la paume.
2. **Zéro surcoût d'inférence** : Seuls les 21 points sont extraits par deep learning ; la classification géométrique s'exécute en moins de 0.1 ms.
3. **Ajustement millimétrique des seuils** : Calibrage interactif possible en direct sur stand.

6.3.3 Hystérésis Temporelle & Stabilité du Tracking Multi-Mains

Pour éliminer les clignotements parasites :

- **Double seuil d'activation/maintien** : Un geste s'active à un score de 0.65, mais ne se désactive que s'il redescend sous 0.45.
- **Temporisation asymétrique** : Activation après 60 ms de confirmation continue, et persistance maintenue pendant 250 ms après perte de repère.
- **Association par matrice de coût (`track_id`)** : Chaque main détectée se voit attribuer un identifiant persistant basé sur la proximité euclidienne de la paume, l'échelle et la latéralité.

7 Journal des Pivots & Retours d'Expérience

Cette section consigne les difficultés techniques rencontrées durant le développement, les choix initiaux abandonnés (*pivots*) et les enseignements capitaux.

7.1 Historique des Arbitrages & Pivots Clés

1. **[Pivot] Webcam classique vs Capteur Kinect** :
 - *Idée initiale* : Utiliser une simple webcam USB avec un modèle d'IA de segmentation humaine.
 - *Problème rencontré* : Latence prohibitive (> 60 ms), estimation imprécise de la distance physique, et chute du framerate sous 25 FPS.
 - *Pivot* : Adoption de la Kinect v1. La carte de profondeur native à 30 Hz décharge totalement l'ordinateur de la segmentation et permet un dégradé de couleur analogique selon la profondeur (Z).
2. **[Pivot] Reconnaissance de gestes par ML vs Moteur géométrique custom** :
 - *Idée initiale* : Utiliser le modèle `gesture_recognizer.task` pré-entraîné de Google MediaPipe.
 - *Problème rencontré* : Manque de flexibilité pour inventer nos propres gestes (ex. le pistolet avec détection du pouce armé, ou le retournement rapide de pouce « Gladiator »).
 - *Pivot* : Utilisation de MediaPipe uniquement pour extraire les 21 points 3D de la main, puis classification mathématique propriétaire dans `HandGestureEngine.gd`.

3. [Ajustement] Shader de fond aquatique & Lisibilité :

- *Problème* : Le shader de vagues d'arrière-plan était trop lumineux et noyait visuellement les silhouettes.
- *Ajustement* : Réduction drastique de l'intensité (passage de 0.55 à 0.10) et diminution des ondulations (de 10 à 1) pour créer un fond feutré.

4. [Ajustement] Émission du tracé de dessin :

- *Problème* : Les premières versions émettaient des particules depuis l'ensemble de la main, créant un pâté informe.
- *Ajustement* : Isolement strict du repère du bout de l'index (*Landmark 8*) projeté dans le repère écran avec lissage exponentiel.

7.2 Conseils pour les Futurs Repreneurs

Recommandations pour la Suite du Projet

- **Migration Kinect v2 / Azure Kinect** : Si le matériel peut être renouvelé, migrer vers une Kinect v2 (Xbox One) ou Azure Kinect apporterait une résolution de profondeur en 512×424 ou 1080p, augmentant la finesse des silhouettes.
- **Mode multi-écrans** : Séparer l'affichage public (plein écran sans aucune interface) de l'écran régie (contenant le flux RGB, les FPS et les métriques de tracking) en exploitant le support multi-fenêtres natif de Godot 4.
- **Enregistrement de fresques** : Possibilité d'ajouter une touche pour exporter le dessin des visiteurs sous forme d'image PNG avec QR code à scanner sur smartphone.